Séance 18 : Comprendre le principe de la récursivité

Objectif

Aujourd'hui, on apprend à comprendre une **fonction récursive**.

À la fin de la séance, je dois savoir :

- reconnaître une fonction récursive ;
- repérer le cas d'arrêt ;
- repérer l'appel récursif ;
- suivre un exemple simple pas à pas.

---

1. À retenir

Qu'est-ce qu'une fonction récursive ?

Une fonction récursive est une fonction qui **s'appelle elle-même**.

Idée importante

Au lieu de résoudre un gros problème d'un seul coup, elle :

- traite un cas simple ;
- ou transforme le problème en un problème plus petit.

Deux éléments indispensables

1. Le cas d'arrêt

C'est le moment où la fonction **s'arrête**.

2. L'appel récursif

C'est le moment où la fonction **se rappelle elle-même**.

Formule utile

Récursivité = cas simple + même problème en plus petit

---

2. Exemple guidé

On considère la fonction :

```
def compte_rebours(n):
    if n == 0:
        print("fin")
```

```
    else:
        print(n)
        compte_rebours(n - 1)
```

Questions

1. Quel est le nom de la fonction ?
   .....
2. Quel est son paramètre ?
   .....
3. Quel est le cas d'arrêt ?
   .....
4. Quel est l'appel récursif ?
   .....

---

3. Suivre l'exécution

On appelle :

compte_rebours(3)

Compléter le tableau :

Appel	Test	Action
compte_rebours(3)		
compte_rebours(2)		
compte_rebours(1)		
compte_rebours(0)		

Question

Qu'est-ce qui s'affiche ?
.....
.....
.....
.....

---

4. Exemple avec une valeur renvoyée

On considère la fonction :

```
def somme(n):
    if n == 0:
        return 0
```

```
    else:
        return n + somme(n - 1)
```

Questions

1. Quel est le cas d'arrêt ?
   .....
2. Quel est l'appel récursif ?
   .....
3. Que vaut somme(0) ?
   .....
4. Compléter :

somme(3) = 3 + somme(....)
somme(2) = 2 + somme(....)
somme(1) = 1 + somme(....)
somme(0) = ...

5. Que vaut somme(3) ?
   .....

---

5. Exercices

Exercice 1

```
def etoiles(n):
    if n == 0:
        print("stop")
    else:
        print("*")
        etoiles(n - 1)
```

1. Quel est le cas d'arrêt ?
   .....
2. Quel est l'appel récursif ?
   .....
3. Qu'affiche etoiles(3) ?
   .....
   .....
   .....
   .....

Exercice 2

```
def produit(n):
    if n == 1:
        return 1
```

```
else:
    return n * produit(n - 1)
```

1. Quel est le cas d'arrêt ?
   .....
2. Calculer produit(2) :
   .....
3. Calculer produit(3) :
   .....

---

6. Repérer une erreur

On considère la fonction :

```
def bug(n):
    print(n)
    bug(n - 1)
```

Questions

1. Cette fonction est-elle récursive ?
   .....
2. Quel est le problème ?
   .....
3. Pourquoi cette fonction est-elle dangereuse ?
   .....

---

7. Petit exercice type bac

On considère la fonction suivante :

```
def mystere(n):
    if n == 0:
        return 0
    else:
        return 1 + mystere(n - 1)
```

Questions

1. Pourquoi cette fonction est-elle récursive ?
   .....
2. Quel est son cas d'arrêt ?
   .....
3. Calculer mystere(1) :
   .....

4. Calculer mystere(2) :
   .....
5. Calculer mystere(4) :
   .....
6. Que semble renvoyer cette fonction ?
   .....

---

8. Vrai ou faux ?

a)

Une fonction récursive est une fonction qui s'appelle elle-même.
Vrai / Faux

b)

Une fonction récursive peut fonctionner sans cas d'arrêt.
Vrai / Faux

c)

Dans une fonction récursive, on traite souvent un problème plus petit.
Vrai / Faux

d)

Dans somme(n - 1), on réduit le problème.
Vrai / Faux

---

9. Bilan

Compléter les phrases :

1. Une fonction récursive est
   .....
2. Le cas d'arrêt sert à
   .....
3. L'appel récursif sert à
   .....
4. Le danger principal est
   .....
5. Pour comprendre une récursion, il faut
   .....

CORRECTION

Exemple guidé

1. Nom : compte_rebours
2. Paramètre : n
3. Cas d'arrêt : n == 0
4. Appel récursif : compte_rebours(n - 1)

Suivre l'exécution

Appel	Test	Action
compte_rebours(3)	3 == 0 faux	affiche 3, appelle compte_rebours(2)
compte_rebours(2)	2 == 0 faux	affiche 2, appelle compte_rebours(1)
compte_rebours(1)	1 == 0 faux	affiche 1, appelle compte_rebours(0)
compte_rebours(0)	0 == 0 vrai	affiche fin

Affichage :

3
2
1
fin

Exemple avec une valeur renvoyée

1. Cas d'arrêt : n == 0
2. Appel récursif : somme(n - 1)
3. somme(0) = 0
4. Compléter :

somme(3) = 3 + somme(2)
somme(2) = 2 + somme(1)
somme(1) = 1 + somme(0)
somme(0) = 0

5. somme(3) = 6

Exercice 1

1. Cas d'arrêt : n == 0
2. Appel récursif : etoiles(n - 1)

3. Affichage :

*
*
*
stop

Exercice 2

1. Cas d'arrêt : n == 1
2. produit(2) = 2 * 1 = 2
3. produit(3) = 3 * 2 * 1 = 6

Repérer une erreur

1. Oui, cette fonction est récursive.
2. Il manque un cas d'arrêt.
3. Elle s'appelle sans fin et finit par provoquer une erreur.

Petit exercice type bac

1. Elle est récursive car elle s'appelle elle-même.
2. Cas d'arrêt : n == 0
3. mystere(1) = 1
4. mystere(2) = 2
5. mystere(4) = 4
6. Elle semble renvoyer n.

Vrai ou faux

a) **Vrai**
b) **Faux**
c) **Vrai**
d) **Vrai**

Bilan possible

1. Une fonction récursive est une fonction qui s'appelle elle-même.
2. Le cas d'arrêt sert à arrêter les appels.

3. L’appel récursif sert à traiter un problème plus petit.
4. Le danger principal est d’oublier le cas d’arrêt.
5. Pour comprendre une récursion, il faut suivre les appels étape par étape.

Mini devoir maison

Exercice 1

On considère :

```
def compte(n):
    if n == 0:
        print("stop")
    else:
        print(n)
        compte(n - 1)
```

Qu’affiche compte(2) ?
.....
.....
.....

Exercice 2

On considère :

```
def addition(n):
    if n == 0:
        return 0
    else:
        return n + addition(n - 1)
```

Calculer addition(2) puis addition(4).
.....
.....

Exercice 3

Expliquer en une phrase pourquoi le cas d’arrêt est indispensable.
.....
.....